Instructor Guide — Building and Publishing Your Own JupyterLite Course

This guide explains how to package your course (notebooks + data) into your **own JupyterLite site** and share it with students as a **single link**. Students just click the link: the notebooks and data are already there — no upload, no account, no installation on their side.

> **This course is already published at: https://andreatagliabue.github.io/python-unige/** (built and deployed automatically from the `course_site/content/` folder on every push — see Section 6A). The five lesson notebooks are `01_introduction`, `02_python_basics`, `03_data_io_and_tables`, `04_plotting_and_analysis`, `05_fitting_stats_automation`.

---

1. The big picture

JupyterLite is a static website. Python runs inside the student's browser (via Pyodide/WebAssembly). There is no server doing computation, so:

- you **build the site once** on your computer, with the course material baked in;
- you **host the static files** anywhere (GitHub Pages, Netlify, university web space, ...);
- you share **one URL** with the students.

You have two ways to give students the material:

Approach	What students do	Recommended?
A. Bundle the content	Open the link — notebooks and `data/` are already loaded	✓ **Yes, for this course**
B. Empty JupyterLite	Open the link, then upload notebook + data themselves	Only as fallback

This guide focuses on **Approach A**. (Approach B is what the *student guide* currently describes, in case you ever need it.)

> **Note:** A published JupyterLite link is **public** — anyone with the URL can open it. That is fine for a course, but do not put anything secret in the notebooks.

---

2. One-time setup on your computer

You need Python 3 and `pip` (you already have them). Install the JupyterLite tools, ideally in a dedicated virtual environment so you don't touch your system Python:

```
python3 -m venv ~/jupyterlite-env
source ~/jupyterlite-env/bin/activate

pip install jupyterlite-core jupyterlite-pyodide-kernel
```

- `jupyterlite-core` gives you the `jupyter lite` command.
- `jupyterlite-pyodide-kernel` is the Python kernel that runs in the browser.

> From now on, run the commands below **with this environment active** (`source ~/jupyterlite-env/bin/activate`).

---

3. Organize the course content

Create a folder (e.g. `course_site/`) and inside it a `content/` folder. Everything you put in `content/` becomes the files students see, **with the same structure**. So mirror exactly the layout your notebooks expect:

```
course_site/
└── content/
    ├── 01_introduction.ipynb
    ├── 02_python_basics.ipynb
    ├── ...
    └── data/
        ├── tensile_raw.csv
        ├── hardness_by_treatment.csv
        └── ...
```

> This is the step that solves the data-loading problem: because `data/` sits next to the notebooks inside `content/`, a line like `pd.read_csv("data/tensile_raw.csv")` will just work in the student's browser.

---

4. Build the site

From inside `course_site/`, with the environment active:

```
cd course_site
jupyter lite build --contents content --output-dir dist
```

This produces a complete static website in the `dist/` folder.

---

5. Test it locally (before publishing!)

You **cannot** just double-click the HTML file (browsers block JupyterLite when opened as `file://`). You must serve it over HTTP:

```
python3 -m http.server -d dist 8000
```

Then open **http://localhost:8000** in your browser and check:

1. the notebooks appear in the file browser;
2. the `data/` folder is already there;
3. **Run → Run All Cells** works end to end (the first run is slow — it downloads Python + libraries once).

Press `Ctrl + C` in the terminal to stop the local server.

---

6. Publish it (pick one option)

Option A — GitHub Pages (recommended, free)

The JupyterLite project provides a ready-made template that auto-builds and deploys for you:

1. Go to **https://github.com/jupyterlite/demo** and click **"Use this template" → "Create a new repository"**.
2. In your new repository, put your notebooks and `data/` into the **`content/`** folder (replace the demo files).
3. Commit and push. A **GitHub Action** automatically builds the site and publishes it.
4. In the repo: **Settings → Pages**, make sure Pages is enabled (GitHub Actions source).
5. Your link will be:

   ```
   https://<your-github-username>.github.io/<repository-name>/
   ```

   Share that URL with the students. To update the course later, just edit `content/` and push again.

Option B — Any static host (Netlify, Vercel, university web space)

The `dist/` folder from Section 4 is a normal static website. Upload its contents to any web hosting and share the resulting URL. (Netlify/Vercel let you drag-and-drop the `dist/` folder.)

---

7. Updating the course later

- Edit or add notebooks/data in `content/`, then rebuild (Section 4) or push to GitHub (Section 6A). Students get the new version the next time they open the link.
- Students' own edits live in **their** browser storage. If a student has already opened and modified a notebook, your updated copy may not overwrite their local version

automatically. For a clean refresh they can clear the site data or use the file browser to delete their copy and reload.

---

8. What works and what doesn't

Preinstalled and ready to import (via Pyodide): `numpy`, `pandas`, `matplotlib`, `scipy`, `sympy`, `scikit-learn`, and many more — more than enough for this course.

Extra packages: install them *inside a notebook cell* with:

```
%pip install some-package
```

This only works for pure-Python packages or packages built for Pyodide.

Limitations to keep in mind:

- no system terminal / shell, no GPU;
- heavy computation and very large datasets are slow (everything runs in one browser tab);
- files are stored **per browser, per computer** (not synced) — tell students to download their work (covered in the student guide);
- the published link is public (no access control).

---

9. Quick command reference

```
# one-time setup
python3 -m venv ~/jupyterlite-env
source ~/jupyterlite-env/bin/activate
pip install jupyterlite-core jupyterlite-pyodide-kernel

# build (run from the folder containing content/)
jupyter lite build --contents content --output-dir dist

# test locally
python3 -m http.server -d dist 8000     # then open http://localhost:8000

# publish: push content/ to a repo made from github.com/jupyterlite/demo
```

---

10. Recommendation for this course

Use **Approach A**: bundle every lesson's notebook plus the shared `data/` into `content/`, build, and publish on GitHub Pages. Then the student guide can be simplified to essentially *"open this link and press Run All"* — the whole "create a data folder and upload the files" section becomes unnecessary.

> If you go this route, tell me and I'll trim the student guide (and its PDF) to match the “everything is preloaded” workflow.